Impact of Post-Training Quantization on Encoder-Only Transformer Fairness: A Comprehensive Implementation Protocol

Introduction and Theoretical Context

The deployment of large-scale Transformer models in production environments frequently relies on post-training quantization to mitigate severe latency bottlenecks and memory constraints. As natural language processing architectures have evolved from recurrent neural networks to deep, multi-layered attention mechanisms, the parameter counts have expanded exponentially, necessitating compression techniques for efficient deployment on edge devices and high-throughput servers.[1] Post-training quantization serves this exact purpose by systematically reducing the numerical precision of model weights and activations, typically moving from a standard 32-bit floating-point format down to 16-bit floating-point or 8-bit integer formats.[3]

While the degradation of aggregate task performance—such as overall accuracy or macro F1-scores—is generally considered negligible when employing quantization, an emerging body of critical research highlights a significant vulnerability within this compression paradigm. Specifically, reducing numerical precision can disproportionately erode predictive performance across marginalized demographic subgroups, thereby amplifying algorithmic bias and exacerbating representational disparities.[3] Theoretical investigations into the mechanics of deep neural networks suggest that predictive models inherently prioritize features with high statistical frequency to minimize global loss during optimization.[6] Consequently, the internal representations corresponding to minority demographic groups occupy smaller, less robust volumes within the high-dimensional latent space.[7] When aggressive rounding thresholds are applied during quantization, these minority-associated features are disproportionately likely to collapse or distort, directly precipitating an amplification of demographic disparities without any overt changes to the model architecture.[1]

This report outlines an exhaustive, expert-level protocol designed specifically to guide the execution of a mid-project delivery within a highly constrained four-day timeframe. The primary directive of this project is to isolate and measure the impact of post-training quantization on the fairness and internal representations of encoder-only Transformer models.[3] To achieve a mathematically pure isolation of the quantization effect, this methodology mandates the utilization of a pre-trained BERT encoder operating strictly as a frozen feature extractor, appended with a lightweight, full-precision classifier that is trained entirely

independently.[3] By evaluating this specific architectural configuration across multiple numerical precisions—namely the full-precision baseline, the half-precision variant, and the 8-bit dynamic integer variant—and interrogating the outputs using both group-level fairness metrics and representation-level alignment analyses, the resulting framework provides a highly nuanced, empirically rigorous view of quantization-induced demographic sensitivity.[3]

Foundational Architectures and Model Selection

The fundamental requirement of this analytical framework is to completely decouple the representational shifts caused by gradient-based model fine-tuning from the shifts caused strictly by numerical precision reduction.[3] When a transformer model is fully fine-tuned for a specific downstream classification task, the backpropagation of errors alters the geometric distribution of the embeddings in the latent space across all transformer layers.[11] This parameter updating process inherently confounds the measurement of quantization-induced noise, making it impossible to ascertain whether an observed spike in demographic bias is the result of the fine-tuning optimization surface or the reduced bit-width.[3] Therefore, an encoder-only architecture utilized as a static feature extractor is the optimal mechanism for this investigation.

The BERT-Base-Uncased Architecture

The specific model selected for this protocol is the bert-base-uncased architecture. Originally introduced by researchers at Google, the Bidirectional Encoder Representations from Transformers (BERT) architecture fundamentally altered natural language processing by capturing contextual relationships bidirectionally across a sequence of text.[2] The base variant relies on a 12-layer transformer encoder configuration, featuring a 768-dimensional hidden embedding space and 12 attention heads per layer.[12] The uncased variant is highly preferred for this specific analytical pipeline to standardize inputs, particularly given the inherently noisy and unstandardized nature of online toxicity data and scraped biographical datasets that will be evaluated.[14]

The transformer architecture itself abandons recurrent mechanisms in favor of self-attention, allowing the model to weigh the importance of all words in a sequence simultaneously regardless of their positional distance.[2] In the context of the Jigsaw toxicity dataset, for example, the bidirectional attention mechanism allows the model to understand that the word "black" functions differently in the phrase "black woman" compared to "black heart".[16] Because BERT extracts these deep syntactic and semantic representations during its massive pre-training phase on unlabelled text corpora, the resulting embeddings are exceptionally rich and capable of being utilized directly without further alteration to the encoder weights.[2]

The Frozen Encoder Paradigm

To successfully execute this experiment, the pre-trained BERT encoder must be explicitly

frozen during the task adaptation phase.[3] In the PyTorch deep learning framework, this is achieved by iterating through the model's parameters and explicitly disabling the computational graph tracking mechanism.[9]

Architectural Component	Optimization Status	Precision Status	Function within the Pipeline
Token Embedding Layer	Strictly Frozen (requires_grad=False)	Subject to PTQ Evaluation	Maps vocabulary IDs to initial 768-dimensional dense vectors.
Transformer Blocks (1-12)	Strictly Frozen (requires_grad=False)	Subject to PTQ Evaluation	Applies multi-head self-attention to generate contextual representations.
Linear Classifier Head	Actively Trained (requires_grad=True)	Maintained at FP32	Projects the 768-dimensional pooled output to discrete class logits.

By iterating through the model.parameters() and setting the requires_grad attribute to False, the optimizer is prevented from calculating gradients for the millions of weights contained within the encoder blocks during the backward pass.[11] This ensures that the 768-dimensional representations generated for any given input sequence remain identical throughout the training process, serving as a stable, unchanging bedrock upon which the subsequent classification layers are built.[3]

The sequence representation itself is extracted using the token embedding located at the output of the final hidden layer, known as layer 12.[13] During BERT's original pre-training phase, the token is specifically optimized for Next Sentence Prediction tasks, forcing it to aggregate sequence-level semantics into a single dense vector.[2] This 768-dimensional vector represents the entirety of the sentence's contextual meaning and serves as the sole input to the trainable classification head.[3]

The Trainable Multilayer Perceptron

The final component of the architecture is a shallow Multilayer Perceptron, commonly referred to as a linear classification head.[3] This component typically consists of a dropout layer for regularization—preventing the classifier from overfitting to specific dimensions of the static BERT embeddings—followed by a dense linear projection layer that maps the 768-dimensional `` embedding to the target class logits.[9] During the subsequent experimentation phases, it is imperative that this lightweight classifier remains in 32-bit full precision across all trials. By strictly varying the numerical precision of the frozen encoder while keeping the classifier training and inference in full precision, the analysis guarantees that any observed performance degradation or fairness disparity stems exclusively from the encoder's quantization process.[3]

Data Acquisition and Preprocessing Protocols

To evaluate algorithmic fairness comprehensively, the analysis cannot rely on a single domain or a single definition of bias. True fairness evaluations must span multiple dimensions of demographic disparity, including direct correlations with toxic language, the perpetuation of societal stereotypical associations in professional contexts, and strict counterfactual robustness at the individual grammatical level.[3] Consequently, the protocol mandates the acquisition and rigorous preprocessing of three distinct datasets, each serving a highly specialized role in isolating the effects of quantization on model representations.

The Jigsaw Unintended Bias in Toxicity Classification Corpus

The Jigsaw dataset provides the empirical foundation for evaluating group-level fairness metrics, specifically concerning the disproportionate flagging of minority identities as toxic.[3] Developed initially for a Kaggle competition hosted by the Conversation AI team, this corpus consists of nearly two million comments sourced from the defunct Civil Comments platform, explicitly annotated for toxicity and various demographic identities.[16]

The primary machine learning task formulated for this dataset is binary classification, categorizing text as either Toxic or Non-Toxic.[3] However, the raw target variable provided in the dataset is not binary; rather, it is a continuous fractional value between 0.0 and 1.0 representing the aggregate proportion of human annotators who deemed the comment to be toxic.[21] To execute the classification task, this continuous target must be binarized using a strict threshold.[24] The established standard for this specific dataset dictates that any comment with a target score of ≥ 0.5 is assigned to the positive, toxic class, while scores below this threshold belong to the negative class.[24]

Crucially for the fairness evaluation, a substantial subset of these comments includes fractional annotations for a wide array of sensitive identity attributes.[21] These attributes encompass race (e.g., black, white, asian), religion (e.g., christian, jewish, muslim), sexual orientation (e.g.,

homosexual_gay_or_lesbian, bisexual), gender (e.g., male, female, transgender), and disability status (e.g., psychiatric_or_mental_illness).[21] Similar to the toxicity target, these identity annotations must be binarized to create the discrete categorical subgroups required for the mathematical calculation of parity metrics.[24]

Acquiring this dataset presents specific technical challenges. The HuggingFace datasets library standardly provides access via the google/jigsaw_unintended_bias identifier.[19] However, due to recent security updates restricting arbitrary Python code execution within dataset loading scripts, the default viewer and loading functions are frequently disabled.[21] The recommended workaround involves utilizing the convert_to_parquet automated data support function within the HuggingFace ecosystem, or alternatively, utilizing the Kaggle Command Line Interface to download the raw CSV files directly into the computational environment.[19]

The Bias in Bios Professional Corpus

While the Jigsaw dataset addresses overt toxicity, the Bias in Bios dataset is deployed to analyze the impact of quantization on the more subtle phenomenon of semantic representation bias and occupational stereotyping.[3] Introduced by De-Arteaga et al. in 2019, this dataset consists of hundreds of thousands of biographical texts scraped from online platforms, with each biography annotated for one of 28 distinct professional occupations, ranging from Surgeon and Physician to Photographer and Professor.[29]

The machine learning objective for this corpus is multi-class occupation classification, utilizing the biographical text to predict the individual's profession.[3] The explicit protected attribute embedded within this dataset is binary gender, which is generally inferred from the explicit pronouns used throughout the biographical descriptions.[28]

The immense utility of this dataset for fairness evaluations lies in the severe gender imbalances present across the various occupational categories, which accurately reflect historical societal biases and labor market demographics.[29] Because machine learning models seek out the most statistically predictive features, a standard NLP classifier will rapidly learn to associate female pronouns with female-dominated professions (such as nursing) and male pronouns with male-dominated professions (such as engineering).[33] By measuring the True Positive Rates for both male and female instances within the exact same occupational category, the analytical framework can identify whether the process of quantization exacerbates the model's reliance on these gendered proxy variables to achieve its predictive accuracy.[3]

The Equity Evaluation Corpus for Counterfactual Robustness

To completely isolate individual-level fairness and to calculate representational shift without the interference of contextual confounding variables, the Equity Evaluation Corpus serves as a perfectly controlled syntactic laboratory.[3] Traditional datasets like Jigsaw and Bias in Bios suffer from uncontrollable variance; sentences containing male pronouns often differ entirely in topic,

vocabulary, and sentiment from sentences containing female pronouns.[19]

The Equity Evaluation Corpus resolves this by providing 8,640 synthetic English sentences generated from a rigid set of fixed semantic templates.[36] These templates are designed such that they vary strictly by a single demographic identity term—for instance, swapping a traditionally European-American name for a traditionally African-American name, or substituting "he" for "she"—while maintaining absolutely identical contextual semantics surrounding the target word.[38]

Because the semantic context is held mathematically constant, any divergence in the model's predicted probability or discrete output class across a demographic swap serves as a direct, irrefutable indicator of algorithmic bias.[19] Furthermore, executing these strictly paired sentences through both the full-precision baseline model and the quantized integer model allows for the precise calculation of layer-wise representation drift.[3] By measuring exactly how much the internal embeddings shift when the only variable introduced is quantization noise on a static sentence structure, researchers gain unprecedented insight into the physical mechanics of bias amplification.[3]

Corpus Designation	**Primary Optimization Objective**	**Evaluated Protected Attributes**	**Target Fairness Application**
Jigsaw Unintended Bias	Binary Toxicity Detection	Race, Religion, Gender, Sexual Orientation	Demographic Parity, Equal Opportunity
Bias in Bios	28-Class Occupation Prediction	Binary Gender (Pronoun Inference)	Equal Opportunity Difference
Equity Evaluation Corpus	Synthetic Counterfactual Generation	Race, Gender (Names & Pronoun Swaps)	Counterfactual Flip Rate, CKA Alignment

Full-Precision (FP32) Training and Optimization

Mechanics

The successful execution of the full-precision baseline phase dictates the fundamental integrity of the subsequent quantization analysis. If the baseline model fails to converge or exhibits severe overfitting, the quantization noise will be applied to unstable representations, rendering the fairness comparisons statistically invalid. The training protocol focuses exclusively on optimizing the appended Multilayer Perceptron while carefully managing the high-dimensional outputs of the frozen BERT encoder.[3]

PyTorch Implementation and Forward Pass Mechanisms

The implementation process initiates by instantiating the pre-trained BertModel utilizing the HuggingFace transformers library, which provides a direct interface to PyTorch.[42] A critical requirement for this specific experimental design is the ability to extract the internal representations from every layer of the transformer for the subsequent Centered Kernel Alignment analysis.[3] To facilitate this, the model must be initialized with the configuration parameter output_hidden_states=True.[44] This parameter overrides the default forward pass behavior, forcing the model to return a complex tuple containing 13 distinct tensors: the output of the initial token embedding layer, followed sequentially by the hidden state outputs of all 12 transformer encoder blocks.[44]

The forward pass is constructed to process tokenized input strings, which have been padded or truncated to a standardized maximum sequence length (typically 128 or 256 tokens to balance memory constraints with contextual coverage).[47] The tokenizer generates two primary inputs for the model: the input_ids, which represent the integer vocabulary indices of the tokens, and the attention_mask, which informs the self-attention mechanism to ignore artificially added padding tokens.[12]

Upon receiving these inputs, the frozen BERT layers execute their matrix multiplications, yielding the aforementioned tuple of hidden states.[13] The protocol dictates that the embedding corresponding to the token from the final hidden layer is isolated.[3, 49] Because the token is perpetually located at the 0th index of the sequence dimension, it is programmatically extracted using tensor slicing methodologies, specifically targeting last_hidden_state[:, 0, :] to yield a batch of 768-dimensional vectors.[13] This vector batch is then passed through the dropout regularization layer and finally through the linear classification head to produce the unnormalized predictive logits.[9]

Hyperparameter Configuration and Optimization Dynamics

Because only the final linear layer (possessing dimensions of $768 \times N_{classes}$) is actively accumulating gradients and undergoing parameter updates, the computational overhead and memory requirements are drastically reduced compared to a full fine-tuning paradigm.[11] This

structural reality fundamentally alters the optimal hyperparameter configurations required for convergence.

The standard optimization algorithm utilized for transformer models is AdamW, which incorporates weight decay for enhanced regularization.[15] However, the learning rate must be carefully adjusted. In a full fine-tuning scenario where all 110 million parameters of the BERT base model are updated, a highly conservative learning rate (e.g., 2×10^{-5}) is required to prevent the phenomenon of catastrophic forgetting, wherein the model irreversibly corrupts its pre-trained linguistic knowledge.[11] Conversely, when optimizing only a randomly initialized linear classification head built upon frozen embeddings, the learning rate must be substantially higher—typically in the range of 1×10^{-3} to 2×10^{-3}—to force the linear weights to scale appropriately to the magnitude of the underlying static representations.[15]

The optimization process employs the standard Categorical Cross-Entropy Loss function.[41] A severe challenge encountered when training on the Jigsaw and Bias in Bios datasets is intense class imbalance; for instance, toxic comments constitute a very small minority of the overall corpus, and certain professions in the Bias dataset contain vastly more samples than others.[25] If unaddressed, the model will rapidly converge on a sub-optimal local minimum by simply predicting the majority class for every input.[31] To counteract this, the loss function must be instantiated with explicit class weights inversely proportional to the class frequencies within the training distribution, thereby penalizing the misclassification of minority instances heavily and ensuring a balanced learning trajectory.[16]

Given the shallow nature of the trainable parameters, convergence is generally observed rapidly. The training loop typically requires no more than 5 to 10 epochs over the dataset to reach optimal validation accuracy and macro F1-scores.[9] Once convergence is achieved, the full-precision weights of the classification head are saved to disk, establishing the permanent FP32 baseline against which all subsequent quantization regimes will be evaluated.[3]

Post-Training Quantization (PTQ) Paradigms

Following the establishment of the full-precision baseline, the experimental protocol advances to the core intervention: the application of Post-Training Quantization to the frozen BERT encoder.[3] Post-training quantization represents a highly efficient deployment strategy that systematically reduces the memory footprint and accelerates the inference latency of deep neural networks without necessitating a computationally expensive retraining or quantization-aware training cycle.[1]

The Mathematics of Quantization Mapping

At its core, quantization is an information compression technique.[1] The mathematical objective is to map a continuous spectrum of 32-bit floating-point values into a highly restricted, discrete

set of integer bins—most commonly the 256 distinct values available within an 8-bit integer format.[1] This mapping is almost universally achieved through an affine linear transformation.

The conversion from a floating-point tensor x to a quantized integer tensor x_q is governed by the equation:

$$x_q = \text{round}\left(\frac{x}{S} + Z\right)$$

where S represents a floating-point scale factor that determines the step size between representable integers, and Z represents an integer zero-point that aligns the real-world value of zero with a specific integer bin, ensuring that zero-padding operations do not introduce asymmetric numerical errors.[1] To recover an approximation of the original floating-point value for downstream calculations, the inverse dequantization function is applied:

$$x_{approx} = (x_q - Z) \times S$$

The fundamental issue underlying all quantization-induced bias is that x_{approx} is rarely identical to the original x.[1] The rounding operation introduces permanent, irreversible quantization noise into the numerical representations, a phenomenon that compounds as the data flows through the successive layers of the transformer architecture.[3]

Dynamic Range Quantization Implementation

For Transformer architectures like BERT, the most efficacious implementation strategy is Dynamic Range Quantization.[51] Throughput in the massive feed-forward and attention projection layers of a transformer is predominantly bottlenecked by the sheer memory bandwidth required to load gigabytes of weight matrices from storage into the processor's active memory.[4]

Dynamic range quantization addresses this specific bottleneck elegantly.[51] During the offline conversion process, the floating-point weights of designated computational layers—specifically the vast nn.Linear modules that dominate the BERT architecture—are statically quantized to 8-bit integers and stored permanently in this compressed format.[10] However, the dynamic aspect dictates that the network's activations (the data flowing between the layers) remain in a floating-point format during memory storage and transfer.[10]

Just prior to the execution of a matrix multiplication operation during the forward pass, the arriving floating-point activations are dynamically observed, scaled, and quantized to 8-bit integers on the fly.[4] The core matrix multiplication is then executed utilizing highly optimized, high-throughput integer arithmetic logic units.[1] Immediately following the multiplication, the

resulting integer output is dequantized back into a floating-point format before being passed to the next layer.[53]

Within the PyTorch framework, this complex orchestration is executed seamlessly utilizing the torch.quantization.quantize_dynamic API.[10] The application of this technique yields a nearly 4x reduction in the encoder's parameter memory footprint and typically generates a 1.8x to 2.5x acceleration in inference speed on compatible CPU and hardware accelerator architectures, satisfying the core industrial motivations for utilizing quantization.[51]

Half-Precision (FP16) Stratification

To accurately map the trajectory of fairness degradation and establish whether bias amplification scales linearly or non-linearly with precision reduction, the protocol mandates the inclusion of an intermediate 16-bit floating-point (FP16) baseline.[3] FP16 quantization operates differently than integer quantization; rather than mapping to discrete integer bins via scale factors, it simply truncates the mantissa (the fractional component) of the standard IEEE 754 floating-point representation, effectively reducing the memory footprint by half while maintaining a massive dynamic range.[4]

In PyTorch, this half-precision stratification is instantiated either by permanently casting the model's parameters using the .half() method, or by wrapping the forward pass execution within a torch.cuda.amp.autocast() context manager.[50] This allows researchers to observe the specific behavioral shifts that occur prior to the severe truncation of the 8-bit integer regime.

The Theoretical Mechanics of Quantization-Induced Bias

The intersection of numerical precision and algorithmic fairness relies on the geometric realities of the latent space. Because predictive machine learning models are fundamentally optimized to minimize aggregate loss across an entire training corpus, they naturally prioritize the robust mathematical representation of features with high statistical frequency.[3] Consequently, the internal representations that correspond to minority demographic groups, less frequent linguistic structures, or nuanced contextual dependencies occupy significantly smaller, less distinct volumes within the high-dimensional manifold.[6]

When the aggressive rounding thresholds and scale factors of integer quantization are applied to this manifold, the mathematical noise is not uniformly destructive.[1] The vast, robust regions of the latent space representing majority features possess sufficient structural integrity to withstand the rounding errors, allowing the classifier to maintain high accuracy on the majority of the dataset.[7] However, the fragile, low-frequency representations associated with marginalized groups are disproportionately likely to be collapsed, merged, or entirely distorted by the quantization noise.[3] This asymmetric representational collapse forces the downstream linear classifier to rely on spurious correlations and heavily biased proxy variables, directly precipitating the measurable amplification of demographic disparities observed in low-bit

regimes.[3]

Group and Individual Fairness Evaluation Frameworks

To empirically validate the theoretical mechanics of quantization-induced bias and determine precisely how reduced precision alters demographic sensitivity, a multi-faceted evaluation framework is critical.[3] Standard performance metrics such as accuracy and F1-scores operate on aggregate populations and are mathematically blind to localized demographic disparities.[55] Therefore, the protocol leverages the specialized algorithms of the fairlearn Python library, alongside custom scripting to execute counterfactual robustness checks.[55]

Group-Level Evaluation: Demographic Parity Difference

Demographic Parity asserts a strict definition of fairness: the likelihood of a machine learning model predicting a positive outcome must be statistically independent of the subject's protected attribute.[55] In the context of the Jigsaw dataset, this dictates that if the classifier flags 10% of comments referencing Group A (e.g., heterosexual individuals) as toxic, it must also flag approximately 10% of comments referencing Group B (e.g., homosexual individuals) as toxic.[55]

The primary metric utilized to quantify this behavior is the Demographic Parity Difference (DPD). It is calculated mathematically as the absolute difference between the largest and the smallest group-level selection rates across all values of the sensitive feature [58]:

$$DPD = |P(\hat{Y} = 1|A = 0) - P(\hat{Y} = 1|A = 1)|$$

Where $\hat{Y}$ represents the model's predicted label and A denotes the protected demographic attribute.[59] An ideal, completely unbiased model yields a DPD of 0.[58] Utilizing the PyTorch and Fairlearn ecosystems, this metric is computed seamlessly via the fairlearn.metrics.demographic_parity_difference API.[58] The function requires the ground truth labels, the model's predicted labels, and an array containing the discrete sensitive feature annotations for each data point.[58] By comparing the DPD of the full-precision baseline against the DPD of the INT8 variant, researchers can definitively state whether quantization has actively eroded selection parity.[3]

Group-Level Evaluation: Equal Opportunity Difference

While Demographic Parity demands equal selection rates regardless of ground truth, Equal Opportunity is a slightly relaxed variant that evaluates whether the model yields identical True Positive Rates (TPR) across demographic subgroups.[55] This metric is profoundly relevant for the Bias in Bios dataset.[35] Equal Opportunity dictates that an individual who is actually qualified to be a surgeon should be correctly classified as a surgeon at the exact same rate, regardless of

whether their biography utilizes male or female pronouns.[35]

The Equal Opportunity Difference (EOD) is defined mathematically as the absolute disparity between the true positive rates of the highest-performing and lowest-performing demographic groups [3]:

$$EOD = |TPR_{A=0} - TPR_{A=1}|$$

$$EOD = |P(\hat{Y} = 1|A = 0, Y = 1) - P(\hat{Y} = 1|A = 1, Y = 1)|$$

Using the fairlearn.metrics.equal_opportunity_difference function, the analytical framework isolates the model's classification performance strictly on the positive ground-truth instances, ignoring false positives, and measures the maximum disparity across the subgroups.[60] The EOD metric is extraordinarily sensitive to representational collapse.[59] If the integer quantization process destroys the subtle, non-gendered syntactic features that differentiate male and female biographies within a specific profession, the model will fall back on gendered proxies, causing the EOD to spike severely in the INT8 evaluation.[31]

Individual Fairness: The Counterfactual Flip Rate

While DPD and EOD measure aggregate population statistics to identify broad societal biases, they fail to capture localized prediction instability at the individual grammatical level.[55] Counterfactual fairness requires that a model's specific prediction for a unique input instance remains entirely invariant if the protected demographic attribute within that input is altered.[19]

To evaluate this, the protocol utilizes the Equity Evaluation Corpus (EEC).[3] Paired synthetic sentences—such as x_i : "Alonzo feels angry" versus its counterfactual x_i^{swap} : "Adam feels angry"—are passed through the network sequentially.[3] The Counterfactual Flip Rate (CFR) is then calculated as the empirical fraction of test cases where the singular demographic substitution forces the predicted class $\hat{y}$ to completely flip [3]:

$$CFR = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(\hat{y}(x_i) \neq \hat{y}(x_i^{swap}))$$

Where N is the total number of paired sentences and $\mathbb{I}$ represents the indicator function.[3] If the INT8 quantized model exhibits a statistically significant higher Counterfactual Flip Rate than the FP32 baseline model, it provides irrefutable evidence that the reduced numerical precision has inadvertently increased the model's reliance on specific demographic proxy tokens to formulate its predictions.[56]

Fairness Metric	Mathematical Objective	Ideal Value	Relevant Evaluation Dataset
Demographic Parity Difference	Equality of positive selection rates across groups.	0.0	Jigsaw Unintended Bias
Equal Opportunity Difference	Equality of True Positive Rates (TPR) across groups.	0.0	Bias in Bios
Counterfactual Flip Rate	Prediction invariance under demographic substitution.	0.0%	Equity Evaluation Corpus

Representation-Level Interrogation and Alignment Metrics

Identifying that a model has become more biased after quantization satisfies only half of the analytical objective. To understand exactly *where* and *why* this fairness degradation occurs physically within the network, the internal representations of the Transformer must be interrogated.[3] By extracting the massive hidden_states tuple—containing the outputs from all 12 attention layers—for a fixed, deterministic sample of inputs, the framework can calculate the specific geometric divergence injected by the quantization algorithm.[44]

L2 Distance and Cosine Similarity Shifts

For any given transformer layer denoted as l, the absolute geometric distortion between the highly accurate full-precision representation h_l^{FP32} and the compressed quantized representation h_l^{Quant} is measured using the standard L2 Norm.[3] This metric quantifies the raw physical distance that the embedding vectors have been displaced within the latent space due to integer rounding [3]:

$$\Delta_l^{(2)} = \frac{1}{N}\sum_{i=1}^{N} ||h_l^{FP32}(x_i) - h_l^{Quant}(x_i)||_2$$

However, absolute distance does not tell the complete story. In deep learning embeddings, the directional orientation of a vector often encodes its core semantic meaning, while its magnitude encodes intensity.[13] Therefore, to assess whether quantization actively alters the semantic orientation of the embeddings, the Cosine Similarity shift is computed [3]:

$$cos_l = \frac{1}{N}\sum_{i=1}^{N} \frac{\langle h_l^{FP32}(x_i), h_l^{Quant}(x_i)\rangle}{||h_l^{FP32}(x_i)||_2 \cdot ||h_l^{Quant}(x_i)||_2}$$

Because deep neural networks process information sequentially, errors compound. It is expected that deeper layers in the BERT architecture will accumulate substantially more quantization noise than the early embedding layers. Tracking the L2 and Cosine metrics layer-by-layer allows researchers to pinpoint the specific transformer blocks that act as critical failure points for demographic fairness.[3]

Centered Kernel Alignment (CKA) Analysis

While L2 and Cosine similarities operate on individual vectors, Centered Kernel Alignment (CKA) represents a profound advancement in representation analysis by measuring the macroscopic similarity of the entire representational structure across entire layers and manifolds.[7] Crucially, CKA is mathematically invariant to orthogonal transformations and isotropic scaling, making it the absolute gold standard for comparing feature maps across divergent networks or, in this case, divergent numerical precision states.[3]

The mathematical formulation of CKA relies on the computation of Gram matrices. Given two representation matrices X and Y of shape $N \times d$ (where N is the batch size of inputs and d is the 768-dimensional embedding space), we define the Gram matrices $K = XX^T$ and $L = YY^T$.[66] Linear CKA is then calculated via the Hilbert-Schmidt Independence Criterion (HSIC), which measures the dependence between these two matrices.[66]

The raw HSIC computation involves a centering matrix H, yielding the formulation $HSIC_0(K, L) = \frac{tr(KHLH)}{(n-1)^2}$.[66] To obtain the final, normalized CKA score, the HSIC value is scaled by the auto-alignment of the matrices:

$$CKA(K, L) = \frac{HSIC(K, L)}{\sqrt{HSIC(K, K) \cdot HSIC(L, L)}}$$

Standardizing this massive matrix calculation across the massive embedding spaces of a

transformer requires specialized techniques to avoid out-of-memory errors on typical hardware. An unbiased minibatch estimator for CKA, denoted as $HSIC_1$, removes the dependency on total batch size and can be utilized via PyTorch-native libraries such as ckatorch.[66] By calculating this alignment metric for each of the 12 transformer layers, researchers gain a high-fidelity map of structural integrity. A high CKA score (approaching 1.0) indicates that the quantized layer perfectly preserves the relational geometry of the high-precision layer.[43] Conversely, a severe degradation in CKA scores in the final encoder layers strongly correlates with the emergence of algorithmic bias, proving that the structural collapse of the manifold forces the downstream linear classifier to rely on spurious demographic correlations.[3]

Execution Timeline for Mid-Project Delivery

To transition from an empty codebase to a comprehensively executed, fully documented mid-report featuring the encoder-only models within a strict four-day constraint, execution must follow a highly modular, rigorously scheduled protocol. The following timeline synthesizes all previous theoretical and technical directives into an actionable daily pipeline.

Day 1: Environmental Architecture and Pipeline Engineering

The initial twenty-four hours are dedicated exclusively to establishing the stable technological foundation and securing the complex datasets required for the investigation.

- The primary directive is to initialize a secure Python virtual environment, installing the exact library dependencies required: torch, transformers, datasets, fairlearn, and pandas.[14]
- The data acquisition phase requires downloading the Jigsaw dataset. Due to size and access constraints, utilizing the Kaggle Command Line Interface is recommended. Once downloaded, the fractional toxicity target column must be immediately binarized using the ≥ 0.5 threshold to establish the ground-truth classification labels.[21] The demographic sub-identity columns must be isolated and pre-processed into clean categorical arrays.[21]
- The Bias in Bios corpus and the Equity Evaluation Corpus must be secured. For the EEC, strict data management is required to ensure the counterfactual text pairs are linked by unique, immutable identifiers, which is mandatory for the eventual flip-rate testing.[30]
- Finally, the HuggingFace BertTokenizer is instantiated. Custom PyTorch DataLoader objects are constructed to handle the dynamic padding of sequences and the generation of attention masks for the training and validation splits.[11]

Day 2: Full-Precision Baseline Optimization and Extraction

With the data pipelines verified, the second day is focused on optimizing the trainable classification head and extracting the pure, unquantized representations.

- The FrozenBertClassifier PyTorch module is coded, explicitly setting requires_grad = False

for all internal parameters within the bert-base-uncased layers.[9]

- The classifier is trained independently on the Jigsaw (Toxicity) and Bias in Bios (Occupation) datasets. The training loop must utilize the AdamW optimizer with an elevated learning rate (1×10^{-3}) and a class-weighted Cross-Entropy Loss function to combat dataset imbalance.[31]
- Upon convergence, the extensive validation loop is executed. The overarching Accuracy, Macro F1-Score, and the raw output logits for the FP32 predictions are extracted and persisted to disk.[3]
- Crucially, a fixed, deterministic, randomly seeded sample of 1,000 inputs from the test sets is processed through the network. The 13 hidden_states tensors generated by the output_hidden_states=True configuration are saved to disk, forming the FP32 baseline for the upcoming CKA representation analysis.[44]

Day 3: Quantization Implementation and Fairness Computation

The third day represents the core experimental intervention, introducing quantization noise and measuring the resulting demographic fallout.

- The torch.quantization.quantize_dynamic API is applied to the trained FP32 model, targeting the nn.Linear layers to generate the compressed INT8 variant.[10] Concurrently, the model is cast utilizing the .half() method or autocast context to generate the FP16 variant.[51]
- The complete validation datasets are processed through both the FP16 and INT8 models to collect their respective, noise-injected predictions.[3]
- The fairlearn.metrics library is deployed. For each precision state (FP32, FP16, INT8), the Demographic Parity Difference (DPD) is calculated utilizing the sensitive attributes derived from the Jigsaw dataset, and the Equal Opportunity Difference (EOD) is calculated utilizing the binary gender attributes from the Bias in Bios dataset.[58]
- The paired counterfactual sentences from the Equity Evaluation Corpus are processed through all three models. Custom Python logic calculates the Counterfactual Flip Rate, identifying the exact percentage of predictions that change solely due to demographic pronoun or name substitution.[3]

Day 4: Alignment Analysis and Report Synthesis

The final day bridges the gap between the empirical performance metrics and the physical mechanics of the neural network, concluding with the synthesis of the final report.

- The identical 1,000-sample deterministic dataset utilized on Day 2 is processed through the newly minted INT8 model. The 13 corresponding hidden_states tensors are extracted.[44]
- A mathematical analysis script is executed to compute the layer-wise L2 Distance, Cosine Similarity, and Minibatch CKA scores, directly comparing the FP32 hidden states against

the INT8 hidden states for all 12 transformer blocks.[3]

- Visualizations (line plots) are generated mapping the trajectory of the CKA structural decay and the L2 distance expansion as the data propagates from layer 1 to layer 12.[3]
- The final synthesis involves correlating the magnitude of the CKA structural collapse with the measured spikes in DPD, EOD, and Counterfactual Flip Rates.[3] By proving that representational distortion in the final encoder layers precedes fairness degradation, the resulting report provides a complete, exhaustive, and rigorously supported analysis of quantization-induced bias, fulfilling all requirements for the mid-project submission.

Works cited

1. Practical Quantization in PyTorch, accessed February 28, 2026, https://pytorch.org/blog/quantization-in-practice/
2. How to Code BERT Using PyTorch - Tutorial With Examples - neptune.ai, accessed February 28, 2026, https://neptune.ai/blog/how-to-code-bert-using-pytorch-tutorial
3. ProjectProposal_BERTMen.pdf
4. Accelerate PyTorch Models Using Quantization Techniques - Intel, accessed February 28, 2026, https://www.intel.com/content/www/us/en/developer/articles/code-sample/accelerate-pytorch-models-using-quantization.html
5. (beta) Dynamic Quantization on BERT — PyTorch Tutorials 2.5.0+cu124 documentation, accessed February 28, 2026, https://pytorch-cn.com/tutorials/intermediate/dynamic_quantization_bert_tutorial.html
6. Fairness Metrics in Machine Learning - Coralogix, accessed February 28, 2026, https://coralogix.com/ai-blog/fairness-metrics-in-machine-learning/
7. 1 Introduction - arXiv, accessed February 28, 2026, https://arxiv.org/html/2510.22953v1
8. Impact on bias mitigation algorithms to variations in inferred sensitive attribute uncertainty, accessed February 28, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11924408/
9. Training classifier with frozen DistilBERT embeddings - Beginners - Hugging Face Forums, accessed February 28, 2026, https://discuss.huggingface.co/t/training-classifier-with-frozen-distilbert-embeddings/4000
10. Quantization Recipe — PyTorch Tutorials 2.5.0+cu124 documentation, accessed February 28, 2026, https://pytorch-cn.com/tutorials/recipes/quantization.html
11. Using BERT: Strategies for text classification | by Shubham Kumar Singh | Medium, accessed February 28, 2026, https://medium.com/@shubham.ksingh.cer14/using-bert-strategies-for-text-classification-32f9c211576d
12. A Complete Guide to BERT with Code | Towards Data Science, accessed February 28, 2026,

https://towardsdatascience.com/a-complete-guide-to-bert-with-code-9f87602e4a11/
13. How to extract Sentence Embedding Using BERT model from [CLS] token - Stack Overflow, accessed February 28, 2026, https://stackoverflow.com/questions/68603462/how-to-extract-sentence-embedding-using-bert-model-from-cls-token
14. ceshine/jigsaw-toxic-2019: A solution to the Jigsaw Unintended Bias in Toxicity Classification Kaggle competition. - GitHub, accessed February 28, 2026, https://github.com/ceshine/jigsaw-toxic-2019
15. Toxic Comment Classification using BERT - GeeksforGeeks, accessed February 28, 2026, https://www.geeksforgeeks.org/machine-learning/toxic-comment-classification-using-bert/
16. [Notes] Jigsaw Unintended Bias in Toxicity Classification | by Ceshine Lee - Medium, accessed February 28, 2026, https://medium.com/the-artificial-impostor/notes-jigsaw-unintended-bias-in-toxicity-classification-fd6c8d3e7459
17. How the pytorch freeze network in some layers, only the rest of the training?, accessed February 28, 2026, https://discuss.pytorch.org/t/how-the-pytorch-freeze-network-in-some-layers-only-the-rest-of-the-training/7088
18. MaxPoolBERT: Enhancing BERT Classification via Layer- and Token-Wise Aggregation, accessed February 28, 2026, https://arxiv.org/html/2505.15696
19. Flexible text generation for counterfactual fairness probing - ACL Anthology, accessed February 28, 2026, https://aclanthology.org/2022.woah-1.20.pdf
20. Jigsaw Unintended Bias in Toxicity Classification - Kaggle, accessed February 28, 2026, https://www.kaggle.com/c/jigsaw-unintended-bias-in-toxicity-classification
21. google/jigsaw_unintended_bias · Datasets at Hugging Face, accessed February 28, 2026, https://huggingface.co/datasets/google/jigsaw_unintended_bias
22. google/jigsaw_unintended_bias at main - Hugging Face, accessed February 28, 2026, https://huggingface.co/datasets/google/jigsaw_unintended_bias/blob/main/jigsaw_unintended_bias.py
23. Toxic Comment Classification using Transformer Mod - Kaggle, accessed February 28, 2026, https://www.kaggle.com/code/kayrahanozcan/toxic-comment-classification-using-transformer-mod
24. Jigsaw Unintended Bias in Toxicity Classification - Kaggle, accessed February 28, 2026, https://www.kaggle.com/c/jigsaw-unintended-bias-in-toxicity-classification/data
25. Abrar2652/Unintended-Bias-Toxicity-Detection-Project - GitHub, accessed February 28, 2026, https://github.com/Abrar2652/Unintended-Bias-Toxicity-Detection-Project
26. Jigsaw Unintended Bias in Toxicity Classification - Theseus, accessed February 28, 2026, https://www.theseus.fi/bitstream/handle/10024/226938/Quan_Do.pdf

27. Performing a Fairness Assessment — Fairlearn 0.14.0.dev0 documentation, accessed February 28, 2026, https://fairlearn.org/main/user_guide/assessment/perform_fairness_assessment.html
28. coastalcph/medical-bios · Datasets at Hugging Face, accessed February 28, 2026, https://huggingface.co/datasets/coastalcph/medical-bios
29. [1901.09451] Bias in Bios: A Case Study of Semantic Representation Bias in a High-Stakes Setting - arXiv, accessed February 28, 2026, https://arxiv.org/abs/1901.09451
30. LabHC/bias_in_bios · Datasets at Hugging Face, accessed February 28, 2026, https://huggingface.co/datasets/LabHC/bias_in_bios
31. Simulating a Bias Mitigation Scenario in Large Language Models - arXiv, accessed February 28, 2026, https://arxiv.org/html/2509.14438v1
32. BFH-AMI/BIAS-MitigationFramework - GitHub, accessed February 28, 2026, https://github.com/BFH-AMI/BIAS-MitigationFramework
33. Mitigating Gender Bias in Occupation Classification | by Natasha Butt - Medium, accessed February 28, 2026, https://medium.com/data-science/mitigating-gender-bias-in-occupation-classification-805edb389729
34. Detecting and Evaluating Bias in Large Language Models: Concepts, Methods, and Challenges, accessed February 28, 2026, https://jbds.isdsa.org/jbds/article/view/182/133
35. Fine-tuning LLMs with Cross-Attention-based Weight Decay for Bias Mitigation - ACL Anthology, accessed February 28, 2026, https://aclanthology.org/2025.findings-emnlp.854.pdf
36. Saif | Bias EEC, accessed February 28, 2026, https://saifmohammad.com/WebPages/Biases-SA.html
37. SemEval-2018 Task 1: Affect in Tweets - CodaLab - Competition, accessed February 28, 2026, https://competitions.codalab.org/competitions/17751
38. Examining Gender and Race Bias in Two Hundred Sentiment Analysis Systems - arXiv.org, accessed February 28, 2026, https://arxiv.org/abs/1805.04508
39. The Expanded Equity Corpus, accessed February 28, 2026, https://algorithmicbiaslab.github.io/expanded-equity-corpus/
40. Gender, Race, and Intersectional Bias in Resume Screening via Language Model Retrieval, accessed February 28, 2026, https://arxiv.org/html/2407.20371v1
41. Counterfactual Fairness by Combining Factual and Counterfactual Predictions - NIPS, accessed February 28, 2026, https://proceedings.neurips.cc/paper_files/paper/2024/file/558da25ef295121e2d79209a95e45436-Paper-Conference.pdf
42. PyTorch-Transformers, accessed February 28, 2026, https://pytorch.org/hub/huggingface_pytorch-transformers/
43. c2d-usp/Layer-wise-LoRA-with-CKA - GitHub, accessed February 28, 2026, https://github.com/c2d-usp/Layer-wise-LoRA-with-CKA
44. Hidden states embedding tensors - Transformers - Hugging Face Forums, accessed February 28, 2026,

https://discuss.huggingface.co/t/hidden-states-embedding-tensors/3549
45. How to get all layers(12) hidden states of BERT? · Issue #1827 · huggingface/transformers, accessed February 28, 2026, https://github.com/huggingface/transformers/issues/1827
46. How are contents of hidden_states tuple in BertModel in the transformers library arranged, accessed February 28, 2026, https://stackoverflow.com/questions/61227950/how-are-contents-of-hidden-states-tuple-in-bertmodel-in-the-transformers-library
47. Text classification - Hugging Face, accessed February 28, 2026, https://huggingface.co/docs/transformers/en/tasks/sequence_classification
48. koushik-25/bert-imdb-sentiment - Hugging Face, accessed February 28, 2026, https://huggingface.co/koushik-25/bert-imdb-sentiment
49. How to obtain [CLS] embeddings from fine-tuned BERT model (using Transformers Trainer), accessed February 28, 2026, https://discuss.huggingface.co/t/how-to-obtain-cls-embeddings-from-fine-tuned-bert-model-using-transformers-trainer/19669
50. Quantization-Aware Training for Large Language Models with PyTorch, accessed February 28, 2026, https://pytorch.org/blog/quantization-aware-training/
51. Post-training quantization | Google AI Edge, accessed February 28, 2026, https://ai.google.dev/edge/litert/conversion/tensorflow/quantization/post_training_quantization
52. Introduction to Quantization on PyTorch, accessed February 28, 2026, https://pytorch.org/blog/introduction-to-quantization-on-pytorch/
53. Quantization in Pytorch - Scaler Topics, accessed February 28, 2026, https://www.scaler.com/topics/pytorch/pytorch-quantization/
54. Counterfactual Data Augmentation for Deep Learning Predictive Models - Imperial College London, accessed February 28, 2026, https://www.imperial.ac.uk/media/imperial-college/faculty-of-engineering/computing/public/distinguished-projects/2223-ug-projects/Counterfactual-data-augmentation-for-deep-learning-predictive-models.pdf
55. Common fairness metrics — Fairlearn 0.14.0.dev0 documentation, accessed February 28, 2026, https://fairlearn.org/main/user_guide/assessment/common_fairness_metrics.html
56. google-deepmind/counterfactual_fairness_evaluation_dataset - GitHub, accessed February 28, 2026, https://github.com/google-deepmind/counterfactual_fairness_evaluation_dataset
57. Fairness: Demographic parity | Machine Learning - Google for Developers, accessed February 28, 2026, https://developers.google.com/machine-learning/crash-course/fairness/demographic-parity
58. fairlearn.metrics.demographic_parity_difference, accessed February 28, 2026, https://fairlearn.org/main/api_reference/generated/fairlearn.metrics.demographic_parity_difference.html
59. Analyzing Fairness of Computer Vision and Natural Language Processing Models, accessed February 28, 2026, https://arxiv.org/html/2412.09900v3

60. fairlearn.metrics.equal_opportunity_difference, accessed February 28, 2026, https://fairlearn.org/main/api_reference/generated/fairlearn.metrics.equal_opportunity_difference.html
61. The Sensitivity of Counterfactual Fairness to Unmeasured Confounding - UAI, accessed February 28, 2026, https://auai.org/uai2019/proceedings/papers/213.pdf
62. Counterfactual Logit Pairing - The TensorFlow Blog, accessed February 28, 2026, https://blog.tensorflow.org/2023/04/counterfactual-logit-pairing.html
63. A Comparative Analysis of Counterfactual Explanation Methods for Text Classifiers - arXiv, accessed February 28, 2026, https://arxiv.org/html/2411.02643v1
64. Deciphering Molecular Embeddings with Centered Kernel Alignment - PMC - NIH, accessed February 28, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11482110/
65. Deciphering Molecular Embeddings with Centered Kernel Alignment | Journal of Chemical Information and Modeling - ACS Publications, accessed February 28, 2026, https://pubs.acs.org/doi/10.1021/acs.jcim.4c00837
66. RistoAle97/centered-kernel-alignment: CKA (Centered ... - GitHub, accessed February 28, 2026, https://github.com/RistoAle97/centered-kernel-alignment
67. Building an English Toxic Comment Classifier Using BERT - Pragati Bagul, accessed February 28, 2026, https://bagulpragati.medium.com/toxic-comment-classifier-a992c7369640
68. Fine-Tuning a Hugging Face DistilBERT Model for IMDB Sentiment Analysis, accessed February 28, 2026, https://jamesmccaffrey.wordpress.com/2021/10/29/fine-tuning-a-hugging-face-distilbert-model-for-imdb-sentiment-analysis/